ASSIGNMENT – 4

Software Testing and Quality Assurance

MCA – 2nd Year

Prepared Answers

Name: _____

Roll No.: _____

Department: _____

1. What is a Defect? Explain the importance of defect tracking in software testing.

A defect is a flaw or mistake in software that causes it to behave differently from the specified requirements or expected results. For example, if a shopping application calculates the wrong total price, it contains a defect.

Importance of defect tracking:

- It records defects in one organized place.
- It helps the team understand, assign, and fix problems.
- It prevents defects from being forgotten or duplicated.
- It allows testers to monitor each defect until closure.
- It helps identify repeated problem areas.
- It provides data for improving software quality and future projects.
- It improves communication between testers, developers, and managers.

2. Explain the Defect Life Cycle with a suitable diagram.

The Defect Life Cycle is the sequence of states a defect passes through from the time it is reported until it is closed or otherwise resolved.

A common flow is shown below. The exact states may differ between organizations.

New → Assigned → Open → Fixed/Resolved → Retest → Verified → Closed

|

If still failing

↓

Reopened

|

Assigned/Open again

Other possible outcomes include Rejected, Duplicate, Deferred, and Cannot Reproduce. A tester reports the defect, the team investigates and fixes it, and the tester retests the software. If the fix works, the defect is verified and closed. If the issue remains, it is reopened.

3. What are the different states of a defect in the Defect Life Cycle? Explain each state.

1. New: The defect has been reported and is awaiting review.
2. Assigned: The defect has been allocated to a developer or responsible team.
3. Open: The team accepts the report and investigates the problem.
4. Fixed/Resolved: A change has been made to correct the defect.
5. Ready for Retest: The updated build is available for testing.
6. Retest: The tester checks whether the reported problem has been fixed.
7. Verified: The tester confirms that the correction works.
8. Closed: The defect is considered completed.
9. Reopened: The defect still occurs or the fix is incomplete.
10. Rejected/Invalid: The report is not accepted as a valid defect.

11. Duplicate: The same issue has already been reported.
12. Deferred: The team decides to fix the defect in a later release.
13. Cannot Reproduce: The team cannot recreate the problem with the available details.

Not every tool uses all these states, and some states may have different names.

4. Explain the process of Defect Tracking and Reporting in software testing.

Defect tracking and reporting is the process of recording defects, monitoring their status, and communicating information about them until they are resolved.

Process:

1. Identify the defect while executing tests or reviewing the software.
2. Reproduce the issue and collect evidence such as screenshots or logs.
3. Record the defect in a tracking tool with clear steps and expected and actual results.
4. Classify it by severity and priority.
5. Assign it to the appropriate developer or team.
6. Monitor investigation and fixing progress.
7. Retest the fix when a corrected build is available.
8. Reopen the defect if the problem remains; otherwise, verify and close it.
9. Prepare reports showing defect counts, status, severity, trends, and unresolved risks.

This process improves accountability, communication, and visibility of software quality.

5. What information should be included in a Defect Report? Explain.

A defect report should contain enough information for the team to understand and reproduce the problem.

Important fields:

- Defect ID: Unique reference number.
- Title: Short and clear summary.
- Description: Explanation of the problem.
- Steps to reproduce: Exact steps that cause the defect.
- Expected result: What should happen.
- Actual result: What happened.
- Environment: Build version, operating system, browser, device, and configuration.
- Test data: Inputs or account details needed to reproduce the issue, without exposing sensitive information.
- Severity: The impact of the defect.
- Priority: The urgency of fixing it.
- Attachments: Screenshots, videos, logs, or error messages.
- Reporter and date: Who reported it and when.
- Assignee and status: Person responsible and current state.
- Related requirement or test case: Helps trace the defect to the affected feature.

6. Differentiate between Defect Severity and Defect Priority with suitable examples.

Defect Severity describes how seriously a defect affects the software. Defect Priority describes how urgently the defect should be fixed.

Severity:

- Based on technical or functional impact.
- Usually assessed by the testing or technical team.
- Examples: an application crash is high severity; a small alignment issue is low severity.

Priority:

- Based on urgency, business needs, and release plans.
- Usually decided by the relevant project stakeholders.
- Examples: a spelling mistake on the home page during a major launch may have high priority despite low severity; a serious defect in a rarely used feature may be scheduled later if a safe workaround exists.

In short, severity is about impact; priority is about urgency. A defect can have high severity and high priority, or the two ratings can differ.

7. Explain the Defect Management Process in detail.

Defect Management is the complete process of identifying, recording, classifying, assigning, fixing, tracking, and closing software defects.

Main steps:

1. Defect prevention: Use reviews, standards, and good development practices to reduce defects.
2. Defect identification: Detect problems through testing, reviews, or user feedback.
3. Defect logging: Record the defect with steps, expected and actual results, and evidence.
4. Classification: Categorize the defect and determine severity and priority.
5. Assignment: Send it to the responsible person or team.
6. Analysis: Investigate the cause, impact, and suitable correction.
7. Fixing: Modify the software to remove the defect.
8. Retesting: Check that the original problem has been corrected.
9. Regression testing: Check that the change has not broken other features.
10. Verification and closure: Confirm the fix and close the defect.
11. Reporting and analysis: Review metrics and trends to identify root causes and improve processes.

Effective defect management helps teams control risk and deliver more reliable software.

8. What are the main activities involved in Defect Management? Explain each activity.

The main activities are:

1. Detection: Find defects through test execution, inspections, reviews, or feedback.
2. Logging: Create a detailed record in the defect tracking system.
3. Classification: Categorize the defect by type, module, severity, and priority.

4. Assignment: Allocate the defect to the responsible developer or team.
5. Investigation: Reproduce the issue, examine logs, and identify the root cause.
6. Resolution: Correct the code, configuration, data, or documentation.
7. Retesting: Confirm that the reported defect is fixed.
8. Regression testing: Ensure the correction has not caused new problems.
9. Closure: Close the defect after successful verification.
10. Reporting and prevention: Analyze trends and take steps to avoid similar defects in the future.

9. Explain the role of Defect Management Tools in software testing.

Defect management tools help teams record, organize, assign, track, and report defects. Examples include Jira, Bugzilla, and Azure DevOps.

Their roles include:

- Keeping defect records in a central location.
- Assigning defects to team members and setting deadlines.
- Tracking status changes throughout the defect life cycle.
- Supporting comments, attachments, screenshots, and logs.
- Sending notifications when a defect is updated.
- Linking defects to test cases, requirements, builds, or releases.
- Producing dashboards and reports about defect trends.
- Supporting collaboration among testers, developers, and managers.
- Maintaining history and audit trails.

These tools make defect handling more transparent and reduce the risk of losing important information.

10. What are the important features of a good Defect Tracking Tool? Explain.

A good defect tracking tool should provide:

1. Easy defect creation: Simple forms with required fields.
2. Unique IDs: Every defect can be identified and referenced.
3. Workflow customization: States and transitions can match the team's process.
4. Assignment and notifications: Defects can be allocated and updates communicated.
5. Severity and priority fields: Teams can classify impact and urgency.
6. Search and filtering: Users can quickly find defects by status, owner, module, or release.
7. Attachments: Screenshots, logs, and other evidence can be stored.
8. Reporting and dashboards: Counts, trends, and unresolved defects can be viewed.
9. Traceability: Defects can be linked to requirements, test cases, code changes, and releases.
10. Access control and history: Permissions and change records protect information.
11. Integration: The tool can work with testing, development, and project management tools.
12. Scalability and usability: It should support the team size and remain easy to use.

11. Explain how defects are identified, recorded, assigned, fixed, retested, and closed during the Defect Management Process.

1. Identified: A tester notices that actual software behaviour differs from the expected result.
2. Recorded: The tester creates a defect report containing steps to reproduce, expected and actual results, environment, and evidence.
3. Reviewed and classified: The team checks whether the defect is valid and sets severity and priority.
4. Assigned: The defect is allocated to the developer or team responsible for the affected component.
5. Fixed: The developer investigates the root cause, changes the software, and updates the defect status.
6. Retested: The tester runs the original test again on the corrected build.
7. Regression tested: Related features are checked to ensure the fix has not introduced other defects.
8. Closed or reopened: If the fix works, the tester verifies and closes the defect. If the issue remains, it is reopened and sent back for further work.

12. What are Test Metrics? Explain their importance in software testing.

Test metrics are measurable values used to evaluate testing progress, coverage, quality, and efficiency.

Importance:

- They show how much testing has been completed.
- They help identify gaps in requirements or feature coverage.
- They support estimates for time, cost, and resources.
- They show defect patterns and high-risk areas.
- They help compare planned progress with actual progress.
- They support evidence-based release decisions.
- They help teams improve their testing process over time.

Metrics should be interpreted together with project context and risk. A large number of executed tests alone does not guarantee good quality.

13. Explain different types of Test Metrics used to measure testing activities.

Common test metrics include:

1. Test Case Execution: Number of test cases executed compared with the planned total.
2. Pass Rate: (Passed test cases ÷ Executed test cases) × 100.
3. Fail Rate: (Failed test cases ÷ Executed test cases) × 100.
4. Test Coverage: Percentage of requirements, features, code, or risks covered by tests.
5. Defect Density: Number of defects relative to software size, such as defects per thousand lines of code.
6. Defect Detection Rate: Number of defects found during a particular period or phase.
7. Defect Leakage: Defects discovered after release that were not found during testing.

8. Test Productivity: Number of test cases designed or executed per person or unit of time.
9. Schedule Variance: Difference between planned and actual testing progress or completion.
10. Automation Coverage: Proportion of suitable test cases automated.
11. Test Environment Availability: Amount of planned testing time for which the environment was usable.
12. Mean Time to Repair: Average time needed to fix a defect or restore service, depending on the agreed definition.

14. What is Test Reporting? Explain the importance of test reports in software testing.

Test reporting is the process of communicating testing progress, results, defects, coverage, risks, and quality status to stakeholders.

Importance:

- It gives a clear view of testing progress.
- It shows passed, failed, blocked, and pending tests.
- It highlights serious defects and unresolved risks.
- It helps managers make release and resource decisions.
- It improves communication between testers, developers, and clients.
- It provides a record of testing performed.
- It supports process improvement by showing trends and recurring issues.

15. What are the main contents of a Test Report? Explain.

A test report commonly contains:

1. Report title, project name, build, version, and reporting period.
2. Summary of testing objectives and scope.
3. Testing activities and test types performed.
4. Test environment and configuration.
5. Execution summary: planned, executed, passed, failed, blocked, and not-run tests.
6. Coverage information, including requirement or feature coverage.
7. Defect summary by status, severity, priority, and affected area.
8. Important unresolved defects and risks.
9. Deviations from the test plan and reasons.
10. Comparison of actual results with planned targets.
11. Limitations, assumptions, and areas not tested.
12. Recommendations and release-readiness assessment.
13. Name of the person or team preparing the report and its date.

A useful report is accurate, concise, understandable, and focused on information that helps stakeholders make decisions.

16. Explain different Test Reporting Techniques used in software testing.

Different reporting techniques are used for different audiences and purposes:

1. Summary reports: Give a brief overview of progress, results, and major risks.
2. Detailed test reports: Provide test-by-test results, evidence, and defect references.
3. Dashboards: Show current metrics and status using charts, tables, and indicators.
4. Defect reports: Summarize open, closed, severe, and high-priority defects.
5. Progress reports: Compare completed work with the schedule and planned targets.
6. Trend reports: Show changes over time, such as defect discovery or pass rates.
7. Test coverage reports: Show which requirements, features, or code areas have been tested.
8. Daily or weekly status reports: Communicate recent progress, blockers, and next steps.
9. Automated reports: Testing tools generate results directly after test execution.

Reports should be tailored to their audience: managers usually need summaries and risks, while testers and developers need detailed results.

17. Differentiate between Test Metrics and Test Reports.

Test Metrics:

- Numerical measurements used to evaluate testing.
- Examples include pass rate, defect density, coverage, and execution progress.
- Often represented as counts, percentages, rates, or durations.
- Help measure and compare testing activities.

Test Reports:

- Documents or summaries that communicate test activities and results.
- Include metrics along with explanations, defects, risks, scope, and recommendations.
- Can be daily, weekly, phase-end, or release reports.
- Help stakeholders understand the overall testing status and make decisions.

In short, metrics are measurements; reports organize and explain those measurements with other relevant information.

18. Explain how defect metrics can be used to measure the quality of software.

Defect metrics provide evidence about software stability and the types of problems found. They cannot measure quality perfectly on their own, but they help identify risks.

Examples:

- Defect density: A high value may indicate a problem area, although comparisons must account for software size and complexity.
- Defect severity distribution: Many critical defects suggest significant quality risks.
- Defect leakage: A high number of defects found by users after release may indicate gaps in testing or requirements.
- Defect reopen rate: Frequent reopening may indicate incomplete fixes or unclear requirements.
- Defect removal efficiency: Shows how effectively defects are found and removed before release.
- Defect age: Long-open defects may indicate difficult issues or delays in resolution.
- Defect trends: A rising number of defects late in testing may indicate instability or new changes.

Teams should review these metrics with customer impact, coverage, release scope, and historical data before drawing conclusions.

19. Explain how Test Metrics can be used to measure the effectiveness and efficiency of a testing team.

Test metrics help evaluate whether a team is finding important defects and using its time and resources effectively.

Effectiveness can be assessed through:

- Requirement and risk coverage.
- Important defects detected before release.
- Defect leakage after release.
- Quality and relevance of test cases.
- Defect removal and retest outcomes.

Efficiency can be assessed through:

- Test cases designed or executed per unit of time.
- Time taken to investigate and retest defects.
- Automation coverage and the amount of repeatable work automated.
- Testing effort compared with estimates.
- Time lost because of unavailable environments or test data.

Metrics should not be used in isolation or as simple rankings of individual testers. For example, a tester who finds fewer defects may be working on a stable module. Quality, complexity, risk, and team collaboration must also be considered.

20. How can metrics be used to identify problems and improve testing effectiveness? Explain.

Metrics help teams identify gaps, investigate their causes, and make improvements.

Steps:

1. Set clear goals: Define what success means, such as adequate risk coverage or fewer escaped defects.
2. Collect consistent data: Use agreed definitions and reliable tools.
3. Monitor trends: Compare results across builds, phases, and releases.
4. Identify warning signs: Examples include low coverage, many blocked tests, repeated defect reopening, or increasing defect leakage.
5. Investigate root causes: Check requirements, test design, code changes, environments, and team processes.
6. Take corrective action: Improve test cases, add automation, clarify requirements, stabilize environments, or provide training.
7. Measure again: Compare later results to see whether the change helped.
8. Share findings: Discuss lessons learned with the team and update processes.

Metrics are most useful when they lead to practical action. They should encourage learning and improvement rather than blame, and they should be interpreted in context.